📘 Guía Maestra Definitiva de Examen: CRUD Full-Stack (.NET 10 + Angular)

> **Manual de Estudio y Plantilla Paso a Paso para Evaluaciones Prácticas (90 Minutos)**
> *Ingeniería de Sistemas - UNJFSC | Asignatura: Desarrollo de Sistemas de Información (DSI)*
> *Patrón: Dashboard Administrativo Maestro-Detalle con Barra Lateral (Sidebar de Clase)*

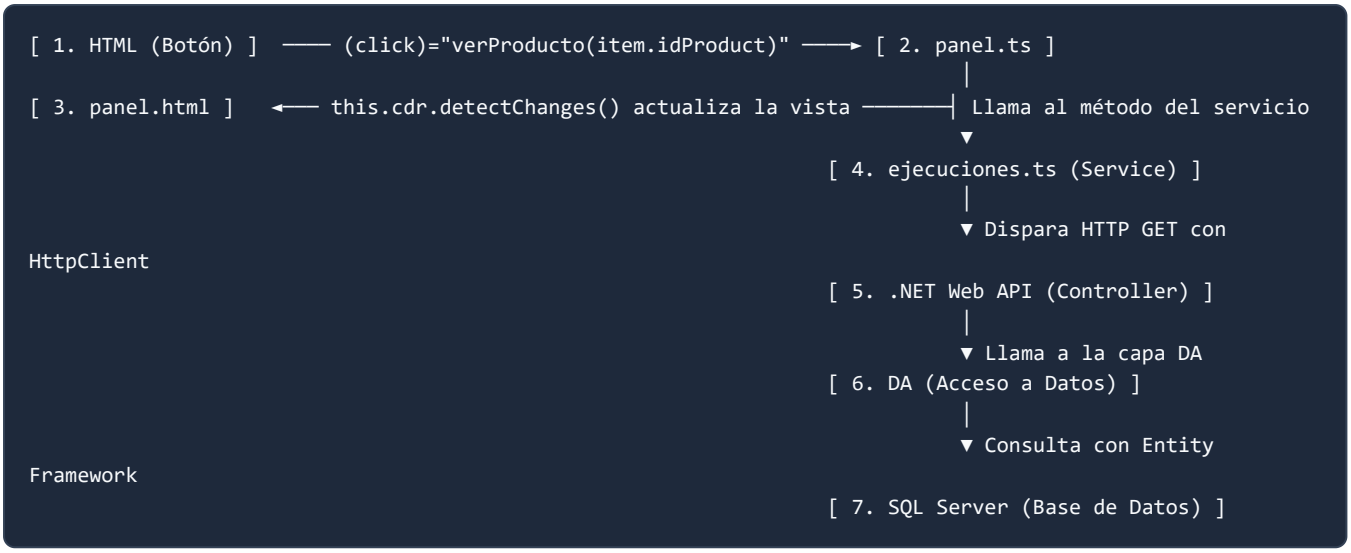
📑 Tabla de Contenidos

1. La Teoría y el Flujo de Conexión: ¿Cómo se comunican las piezas?
2. La Fórmula Camaleón: Cómo adaptar cualquier entidad en 5 minutos
3. Parte 1: Backend (.NET Web API) Simple y Robusto
4. 3.1 Modelos C# (`Product.cs` y `Category.cs`)
5. 3.2 Contexto de Datos (`ApplicationDbContext.cs`)
6. 3.3 Capa de Acceso a Datos (`DAProduct.cs`)
7. 3.4 Controlador API REST (`EjecucionesController.cs`)
8. 3.5 `Program.cs` : Configuración de CORS Obligatoria
9. Parte 2: Frontend (Angular con la Plantilla Exacta de Clase)
10. 4.1 Comandos de Terminal
11. 4.2 Configuración en `app.config.ts` (`provideHttpClient`)
12. 4.3 Modelos TypeScript (`product.model.ts`)
13. 4.4 Servicio HTTP (`ejecuciones.ts`)
14. 4.5 Lógica TypeScript del Componente (`panel.ts`) Comentada
15. 4.6 Vista HTML de Clase (`panel.html`) con Sidebar, Tabla y Ver Detalle
16. 4.7 Estilos CSS de Clase (`panel.css`)
17. 4.8 Conexión en la Raíz (`app.ts` y `app.html`)
18. Parte 3: Los 5 Errores Críticos que Resolvimos y su Solución en 5 Segundos
19. Parte 4: Cronograma de 90 Minutos para el Examen

---

1. La Teoría y el Flujo de Conexión: ¿Cómo se comunican las piezas?

Para no dudar durante el examen, memoriza este **camino del dato**:



¿Por qué se conectan así? (La Lógica)

1. `panel.html` **solo sabe de eventos y variables:**
2. No sabe qué es internet ni qué es SQL. Solo dice: "*Si el usuario hace clic aquí, llama a la función* `verProducto()` *de mi TypeScript*".
3. `panel.ts` **es el Coordinador (El Cerebro):**
4. Recibe la orden del clic.
5. Llama a `this.ejecucionesService.getProductsById(id)`.
6. Como la respuesta tarda unos milisegundos (es asíncrona), usa `.subscribe({ next: (data) => ... })`.
7. Cuando el dato llega, lo guarda en `this.selectProduct = data`, activa `this.mostrarProducto = true` y oculta la tabla con `this.mostrarTabla = false`.
8. **El secreto de ChangeDetectorRef (** `cdr` **):** En versiones modernas de Angular (Zoneless), Angular a veces no se entera de que la respuesta HTTP llegó hasta que haces otro clic. Al poner `this.cdr.detectChanges()`, obligamos a la pantalla a redibujarse en **0 milisegundos**.
9. `ejecuciones.ts` **(Service) es el Mensajero:**
10. Su única responsabilidad es saber las URLs (`https://localhost:7174/api/...`) y los verbos HTTP (`GET`, `POST`, `PUT`, `DELETE`).
11. Devuelve un `Observable`, que es como una encomienda que llegará en unos instantes.
12. `Controller` **de C# es la Puerta de Entrada del Backend:**
13. Expone las rutas públicas (ej: `[HttpGet("ListarPorId/{id}")]`).
14. Lee el parámetro de la URL y se lo entrega al método del `DA`.
15. `DA` **(Data Access) es el Minero:**
16. Abre la conexión con `using var db = new ApplicationDbContext()`.
17. Ejecuta la consulta en SQL Server y retorna la entidad.

---

2. La Fórmula Camaleón: Cómo adaptar cualquier entidad en 5 minutos

En el examen, el profesor puede pedirte cualquier tema: **Productos y Categorías**, **Gatos y Dueños**, o **Detalles y Cabeceras**.

Concepto	Tu Práctica de Hoy	Si te piden Mascotas	Si te piden Ventas
Entidad Padre (Maestra)	`Category`	`Dueno` / `Usuario`	`CabeceraOperacion` / `Cliente`
Entidad Hija (Detalle)	`Products`	`Gato` / `Animal`	`DetalleOperacion` / `Factura`
Clave Primaria Padre	`idCategory`	`idDueno`	`idCabecera`
Clave Primaria Hija	`idProduct`	`idGato`	`idDetalle`
Clave Foránea en la Hija	`idCategory`	`idDueno`	`idCabecera`
Objeto Relacional en Hija	`category?: Category`	`dueno?: Dueno`	`cabecera?: Cabecera`
URL del Backend Hija	`/api/Ejecuciones` (o `/Products`)	`/api/Gatos`	`/api/Detalles`
URL del Backend Padre	`/api/Category`	`/api/Duenos`	`/api/Cabeceras`

> 💡 **Regla de Oro:** Siempre pon la entidad Padre (`Category`, `Dueno`) arriba en tu archivo `model.ts`, y la entidad Hija (`Products`, `Gato`) abajo.

---

3. Parte 1: Backend (.NET Web API) Simple y Robusto

3.1 Modelos C# (`Models/Category.cs` y `Models/Products.cs`)

```
// Models/Category.cs (Entidad Padre)
using System.ComponentModel.DataAnnotations;

namespace BackendWebApp.Models
{
    public class Category
    {
        [Key]
        public int IdCategory { get; set; }

        [Required]
        [MaxLength(100)]
        public string Name { get; set; } = string.Empty;

        public string? Description { get; set; }
    }
}
```

```
// Models/Products.cs (Entidad Hija)
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace BackendWebApp.Models
{
    public class Products
    {
        [Key]
        public int IdProduct { get; set; }

        [Required]
        [MaxLength(100)]
        public string Name { get; set; } = string.Empty;

        public string? Description { get; set; }

        [Required]
        public float Price { get; set; }

        [Required]
        public int Stock { get; set; }

        // Clave Foránea
        public int IdCategory { get; set; }

        [ForeignKey("IdCategory")]
        public virtual Category? Category { get; set; }
    }
}
```

3.2 Contexto de Datos (`Data/ApplicationDbContext.cs`)

```
using BackendWebApp.Models;
using Microsoft.EntityFrameworkCore;

namespace BackendWebApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public DbSet<Category> Category { get; set; }
        public DbSet<Products> Products { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (!optionsBuilder.IsConfigured)
            {

optionsBuilder.UseSqlServer("Server=.;Database=BD_Tienda;Trusted_Connection=True;TrustServerCertificate=Tr
ue;");
            }
        }
    }
}
```

3.3 Capa de Acceso a Datos (`Data/AccesoDatos/DAProduct.cs`)

```
using BackendWebApp.Models;
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using System.Linq;

namespace BackendWebApp.Data.AccesoDatos
{
    public class DAProduct
    {
        // 1. LISTAR TODOS
        public IEnumerable<Products> Listar()
        {
            using var db = new ApplicationDbContext();
            return db.Products.Include(x => x.Category).ToList();
        }

        // 2. OBTENER POR ID (Devuelve un solo objeto con FirstOrDefault)
        public Products? ObtenerPorId(int id)
        {
            using var db = new ApplicationDbContext();
            return db.Products.Include(x => x.Category)
                              .FirstOrDefault(x => x.IdProduct == id);
        }

        // 3. INSERTAR
        public int Insertar(Products entidad)
        {
            using var db = new ApplicationDbContext();
            entidad.Category = null; // Evita conflicto de clave foránea
            db.Products.Add(entidad);
            db.SaveChanges();
            return entidad.IdProduct;
        }

        // 4. ACTUALIZAR
        public bool Actualizar(Products entidad)
        {
            using var db = new ApplicationDbContext();
            var item = db.Products.Find(entidad.IdProduct);
            if (item == null) return false;

            item.Name = entidad.Name;
            item.Description = entidad.Description;
            item.Price = entidad.Price;
            item.Stock = entidad.Stock;
            item.IdCategory = entidad.IdCategory;

            db.SaveChanges();
            return true;
        }

        // 5. ELIMINAR
        public bool Eliminar(int id)
        {
            using var db = new ApplicationDbContext();
            var item = db.Products.Find(id);
            if (item == null) return false;

            db.Products.Remove(item);
            db.SaveChanges();
            return true;
        }
```

```
    }
}
```

3.4 Controlador API REST (`Controllers/EjecucionesController.cs`)

```
using BackendWebApp.Data.AccesoDatos;
using BackendWebApp.Models;
using Microsoft.AspNetCore.Mvc;

namespace BackendWebApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")] // Ruta: api/Ejecuciones
    public class EjecucionesController : ControllerBase
    {
        private readonly DAProduct _da = new DAProduct();

        [HttpGet("Listar")]
        public IActionResult Listar() => Ok(_da.Listar());

        [HttpGet("ListarPorId/{id}")]
        public IActionResult ListarPorId(int id)
        {
            var item = _da.ObtenerPorId(id);
            return item == null ? NotFound() : Ok(item);
        }

        [HttpPost("Insertar")]
        public IActionResult Insertar([FromBody] Products entidad)
        {
            var id = _da.Insertar(entidad);
            return Ok(new { mensaje = "Insertado con éxito", id });
        }

        [HttpPut("Actualizar/{id}")]
        public IActionResult Actualizar(int id, [FromBody] Products entidad)
        {
            entidad.IdProduct = id;
            var ok = _da.Actualizar(entidad);
            return ok ? Ok(new { mensaje = "Actualizado" }) : NotFound();
        }

        [HttpDelete("Eliminar/{id}")]
        public IActionResult Eliminar(int id)
        {
            var ok = _da.Eliminar(id);
            return ok ? Ok(new { mensaje = "Eliminado" }) : NotFound();
        }
    }
}
```

3.5 `Program.cs`: Configuración de CORS Obligatoria

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// 1. DEFINIR LA POLÍTICA CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("PermitirAngular", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyMethod()
              .AllowAnyHeader();
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

// 2. ACTIVAR CORS ANTES DE MAPEAR CONTROLADORES
app.UseCors("PermitirAngular");

app.UseAuthorization();
app.MapControllers();
app.Run();
```

---

4. Parte 2: Frontend (Angular con la Plantilla Exacta de Clase)

4.1 Comandos de Terminal

```
# Crear componente panel
ng g c components/panel --skip-tests

# Crear servicio
ng g s services/ejecuciones --skip-tests
```

4.2 Configuración en `src/app/app.config.ts`

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http'; // 👈 1. IMPORTAR
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient() // 👈 2. ACTIVAR
  ]
};
```

4.3 Modelos TypeScript (`src/app/models/product.model.ts`)

```
// Entidad Padre (Categoría / Dueño / Cabecera)
export interface Category {
  idCategory?: number;
  name: string;
  description?: string;
}

// Entidad Hija (Producto / Gato / Detalle)
export interface Products {
  idProduct?: number;
  name: string;
  description?: string;
  price: number;
  stock: number;
  idCategory?: number;
  category?: Category; // Para mostrar el nombre en la tabla
}
```

4.4 Servicio HTTP (`src/app/services/ejecuciones.ts`)

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Category, Products } from '../models/product.model';

@Injectable({ providedIn: 'root' })
export class EjecucionesService {
  // Ajusta tu puerto según Visual Studio (ej: 7174 o 5264)
  private readonly API_URL = 'https://localhost:7174/api/Ejecuciones';
  private readonly API_CATEGORY_URL = 'https://localhost:7174/api/Category';
  private readonly http = inject(HttpClient);

  // 1. Listar Productos (GET)
  getProducts(): Observable<Products[]> {
    return this.http.get<Products[]>(`${this.API_URL}/Listar`);
  }

  // 2. Obtener Producto por ID (GET)
  getProductsById(id: number): Observable<any> {
    return this.http.get<any>(`${this.API_URL}/ListarPorId/${id}`);
  }

  // 3. Crear Producto (POST)
  createProduct(product: Products): Observable<Products> {
    return this.http.post<Products>(`${this.API_URL}/Insertar`, product);
  }

  // 4. Actualizar Producto (PUT)
  updateProduct(id: number, product: Products): Observable<any> {
    return this.http.put(`${this.API_URL}/Actualizar/${id}`, product);
  }

  // 5. Eliminar Producto (DELETE)
  deleteProduct(id: number): Observable<any> {
    return this.http.delete(`${this.API_URL}/Eliminar/${id}`);
  }

  // 6. Listar Categorías (GET)
  getCategory(): Observable<Category[]> {
    return this.http.get<Category[]>(`${this.API_CATEGORY_URL}/Listar`);
  }
}
```

4.5 Lógica TypeScript del Componente (`src/app/components/panel/panel.ts`)

```
import { Component, inject, ChangeDetectorRef } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { EjecucionesService } from '../../services/ejecuciones';
import { Products, Category } from '../../models/product.model';

@Component({
  selector: 'app-panel',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './panel.html',
  styleUrls: ['./panel.css']
})
export class Panel {
  // Inyecciones
  private readonly ejecucionesService = inject(EjecucionesService);
  private readonly cdr = inject(ChangeDetectorRef); // 👈 Para refrescar la vista en 0ms

  // Estados de Interfaz
  sidebarContraido = false;
  dashboardAbierto = true;
  usuario = 'Marco Torres (Sistemas)';

  // Datos en memoria
  products: Products[] = [];
  category: Category[] = [];
  selectProduct: Products | null = null;

  // Banderas de control de vistas
  mostrarTabla = false;
  mostrarProducto = false;
  cargando = false;
  error = '';

  toggleSidebar(): void {
    this.sidebarContraido = !this.sidebarContraido;
  }

  toggleDashboard(): void {
    this.dashboardAbierto = !this.dashboardAbierto;
  }

  // 1. CARGAR LISTA DE PRODUCTOS
  cargarProductoEjecucion(): void {
    this.cargando = true;
    this.error = '';
    this.mostrarTabla = false;
    this.mostrarProducto = false;

    this.ejecucionesService.getProducts().subscribe({
      next: (data) => {
        this.products = data;
        this.mostrarTabla = true;
        this.cargando = false;
        this.cdr.detectChanges(); // 👈 Actualiza en 0ms
      },
      error: (err) => {
        console.error('Error:', err);
        this.error = 'No se pudo cargar la lista de Productos';
        this.cargando = false;
        this.cdr.detectChanges();
      }
```

```
    });
  }

  // 2. VER DETALLE DE UN PRODUCTO POR ID
  verProducto(idProduct?: number): void {
    if (!idProduct) return; // 👈 Evita error de undefined

    this.cargando = true;
    this.error = '';

    this.ejecucionesService.getProductsById(idProduct).subscribe({
      next: (data: any) => {
        // Blindaje: si el backend devuelve [ { ... } ], toma la posición 0
        this.selectProduct = Array.isArray(data) ? data[0] : data;
        this.mostrarProducto = true;
        this.mostrarTabla = false;
        this.cargando = false;
        this.cdr.detectChanges(); // 👈 Actualiza en 0ms
      },
      error: (err) => {
        console.error('Error:', err);
        this.error = 'No se pudo obtener el detalle del Producto';
        this.cargando = false;
        this.cdr.detectChanges();
      }
    });
  }

  // 3. VOLVER AL LISTADO
  volverListado(): void {
    this.mostrarProducto = false;
    this.selectProduct = null;
    this.error = '';
    this.mostrarTabla = true;
    this.cdr.detectChanges(); // 👈 Actualiza en 0ms
  }

  // 4. CARGAR CATEGORÍAS
  cargarCategory(): void {
    this.cargando = true;
    this.error = '';
    this.mostrarTabla = false;
    this.mostrarProducto = false;

    this.ejecucionesService.getCategory().subscribe({
      next: (data) => {
        this.category = data;
        this.mostrarTabla = true;
        this.cargando = false;
        this.cdr.detectChanges();
      },
      error: (err) => {
        console.error('Error:', err);
        this.error = 'No se pudo cargar la lista de Categorías';
        this.cargando = false;
        this.cdr.detectChanges();
      }
    });
  }
}
```

4.6 Vista HTML de Clase (`src/app/components/panel/panel.html`)

```
<div class="admin-layout" [class.sidebar-mini]="sidebarContraido">
    <!-- BARRA LATERAL (SIDEBAR) -->
    <aside class="sidebar">
        <div class="brand">
            <div class="brand-icon">A</div>
            <div class="brand-text">ControlInventario</div>
        </div>
        <div class="sidebar-content">
            <div class="menu-block">
                <div class="menu-item menu-item-main" (click)="toggleDashboard()">
                    <div class="menu-left">
                        <span class="menu-icon">◉</span>
                        <span class="menu-label">DashBoard</span>
                    </div>
                    <span class="menu-arrow">
                        {{dashboardAbierto?'▼':'►'}}
                    </span>
                </div>

                <!-- SUBMENÚS -->
                <div class="submenu" *ngIf="dashboardAbierto">
                    <div class="submenu-item" (click)="cargarProductoEjecucion()">
                        <span class="submenu-icon">◉</span>
                        <span class="submenu-label">Products</span>
                    </div>
                    <div class="submenu-item" (click)="cargarCategory()">
                        <span class="submenu-icon">◉</span>
                        <span class="submenu-label">Category</span>
                    </div>
                </div>
            </div>
        </div>
    </aside>

    <!-- ZONA PRINCIPAL -->
    <div class="main-wrapper">
        <header class="topbar">
            <div class="topbar-left">
                <button class="menu-toggle" (click)="toggleSidebar()">≡</button>
                <a href="#">Home</a>
                <a href="#">Contact</a>
            </div>
            <div class="topbar-right">
                <div class="user-info">
                    <div class="avatar"></div>
                    <span>{{usuario}}</span>
                </div>
            </div>
        </header>

        <main class="content-area">
            <!-- 1. Estado inicial -->
            <div *ngIf="!cargando && !mostrarTabla && !mostrarProducto && !error" class="p-4 text-center">
                <h2>Seleccione una opción del menú</h2>
                <p class="text-muted">Haga clic en <strong>DashBoard ➔ Products</strong></p>
            </div>

            <!-- 2. Cargando -->
            <div *ngIf="cargando" class="p-4 text-center">
                <h2>Cargando la información...</h2>
            </div>
```

```
            <!-- 3. Error -->
            <div *ngIf="error" class="p-4 text-center text-danger">
                <h2>{{ error }}</h2>
            </div>

            <!-- 4. TABLA DE PRODUCTOS -->
            <div class="tabla-container" *ngIf="mostrarTabla && !cargando && !mostrarProducto">
                <table class="tabla" *ngIf="products.length > 0">
                    <thead>
                        <tr>
                            <th align="center">Id</th>
                            <th align="center">Nombre</th>
                            <th align="center">Stock</th>
                            <th align="center">Precio</th>
                            <th align="center">Categoría</th>
                            <th align="center">Acciones</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr *ngFor="let item of products">
                            <td>{{item.idProduct}}</td>
                            <td>{{item.name}}</td>
                            <td>{{item.stock}}</td>
                            <td>S/. {{item.price}}</td>
                            <td>{{item.idCategory}}</td>
                            <td>
                                <button class="btn-editar"><i class="bi bi-pencil-fill"></i></button>
                                <button class="btn-detalle" (click)="verProducto(item.idProduct)"><i
class="bi bi-eye-fill"></i></button>
                                <button class="btn-eliminar"><i class="bi bi-trash-fill"></i></button>
                            </td>
                        </tr>
                    </tbody>
                </table>
            </div>

            <!-- 5. VISTA DE DETALLE DE UN PRODUCTO -->
            <div class="detalle-container p-4" *ngIf="mostrarProducto && selectProduct">
                <div class="detalle-card" style="background: white; border: 1px solid #dee2e6; border-
radius: 8px; padding: 25px;">
                    <h3>Detalle del Producto #{{ selectProduct.idProduct }}</h3>
                    <p><strong>Nombre:</strong> {{ selectProduct.name }}</p>
                    <p><strong>Descripción:</strong> {{ selectProduct.description }}</p>
                    <p><strong>Precio:</strong> S/. {{ selectProduct.price }}</p>
                    <p><strong>Stock:</strong> {{ selectProduct.stock }} unidades</p>
                    <button class="btn-edit btn" style="background: #0d6efd; color: white; margin-top:
15px;" (click)="volverListado()">
                      ⬅ Volver al listado
                    </button>
                </div>
            </div>
        </main>

        <footer class="footer">
            <span>Copyright @2026 <strong>Ingeniería de Sistemas - UNJFSC</strong></span>
        </footer>
    </div>
</div>
```

4.7 Estilos CSS de Clase (`src/app/components/panel/panel.css`)

```
* { box-sizing: border-box; }
.admin-layout { display: flex; min-height: 100vh; background-color: #f4f6f9; font-family: Arial, sans-serif; }
.sidebar { width: 270px; background-color: #343a40; color: #c2c7d0; min-height: 100vh; display: flex; flex-direction: column; transition: width 0.3s ease; }
.brand { display: flex; align-items: center; gap: 12px; padding: 18px 20px; border-bottom: 1px solid #454d55; }
.brand-icon { width: 40px; height: 40px; border-radius: 50%; background-color: #d7dadf; display: flex; align-items: center; justify-content: center; color: #343a40; font-weight: bold; font-size: 1.4rem; }
.brand-text { color: #f8f9fa; font-size: 1.1rem; font-weight: 500; }
.sidebar-content { padding: 14px 10px; flex: 1; }
.menu-item { display: flex; justify-content: space-between; align-items: center; padding: 12px 14px; border-radius: 8px; cursor: pointer; color: #c2c7d0; }
.menu-item:hover { background-color: #3f474e; color: #fff; }
.menu-left { display: flex; align-items: center; gap: 12px; }
.submenu { margin-top: 6px; }
.submenu-item { display: flex; align-items: center; gap: 12px; padding: 12px 16px 12px 26px; color: #c2c7d0; border-radius: 8px; cursor: pointer; }
.submenu-item:hover { background-color: #3f474e; color: #fff; }

/* Modo Contraído */
.admin-layout.sidebar-mini .sidebar { width: 78px; }
.admin-layout.sidebar-mini .brand-text,
.admin-layout.sidebar-mini .menu-label,
.admin-layout.sidebar-mini .menu-arrow,
.admin-layout.sidebar-mini .submenu { display: none !important; }

.main-wrapper { flex: 1; display: flex; flex-direction: column; min-height: 100vh; }
.topbar { background-color: #ffffff; border-bottom: 1px solid #dee2e6; height: 64px; padding: 0 18px; display: flex; justify-content: space-between; align-items: center; }
.topbar-left { display: flex; align-items: center; gap: 22px; }
.menu-toggle { border: none; background: transparent; font-size: 1.2rem; cursor: pointer; color: #495057; }
.user-info { display: flex; align-items: center; gap: 10px; }
.avatar { width: 36px; height: 36px; border-radius: 50%; background: linear-gradient(135deg, #9aa0a6, #6c757d); }
.content-area { flex: 1; background-color: #f4f6f9; padding: 20px; }
.footer { background-color: #ffffff; border-top: 1px solid #dee2e6; padding: 18px 20px; display: flex; justify-content: space-between; color: #495057; }

/* Tabla */
.tabla-container { background: #ffffff; border: 1px solid #dee2e6; border-radius: 6px; overflow-x: auto; box-shadow: 0 1px 3px rgba(0,0,0,0.08); }
.tabla { width: 100%; border-collapse: collapse; background: white; font-size: 14px; }
.tabla th { background: #343a40; color: white; padding: 12px; text-align: left; }
.tabla td { padding: 10px; border-bottom: 1px solid #dee2e6; }
.tabla tr:hover { background: #f1f3f5; }

/* Botones de Acción */
.btn-editar, .btn-detalle, .btn-eliminar { border: none; width: 32px; height: 32px; border-radius: 6px; cursor: pointer; display: inline-flex; align-items: center; justify-content: center; margin: 0 2px; }
.btn-editar { background-color: #0d6efd; color: #fff; }
.btn-detalle { background-color: #ffc107; color: #fff; }
.btn-eliminar { background-color: #dc3545; color: #fff; }
.btn { border: none; padding: 8px 14px; border-radius: 5px; cursor: pointer; }
.detalle-container { max-width: 800px; margin: 0 auto; }
```

4.8 Conexión en la Raíz (`src/app/app.ts` y `src/app/app.html`)

En `src/app/app.ts`:

```
import { Component } from '@angular/core';
import { Panel } from './components/panel/panel';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [Panel],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {}
```

En `src/app/app.html`:

```
<app-panel></app-panel>
```

---

5. Parte 3: Los 5 Errores Críticos que Resolvimos y su Solución en 5 Segundos

1. **La pantalla no se actualiza hasta hacer clic en el Dashboard:**
2. *Causa:* En Angular moderno (Zoneless), la llamada HTTP asíncrona no avisa automáticamente a la vista.
3. *Solución:* Inyectar `ChangeDetectorRef` y llamar a `this.cdr.detectChanges();` dentro del bloque `next: (data) => ...`.
4. **Los datos llegan en la consola pero salen vacíos en la vista:**
5. *Causa:* El Backend devolvió una lista con corchetes `[ { ... } ]` en vez del objeto directo `{ ... }`. Un arreglo no tiene la propiedad `.name`.
6. *Solución:* Usar `this.selectProduct = Array.isArray(data) ? data[0] : data;`.
7. **Error `TS2345: Argument of type 'number | undefined' is not assignable to 'number'`:**
8. *Causa:* Tu modelo tiene `idProduct?: number` (opcional), pero tu función exige obligatoriamente un número.
9. *Solución:* En tu función pon `verProducto(idProduct?: number)` y la primera línea debe ser: `if (!idProduct) return;`.
10. **Error `Property 'productos' does not exist on type 'Panel'. Did you mean 'products'?`:**
11. *Causa:* Desajuste de idioma. En `panel.ts` lo escribiste en inglés (`products`) y en `panel.html` en español (`productos`).
12. *Solución:* Haz que coincidan exactamente en el mismo idioma.
13. **Error `blocked by CORS policy` en el navegador:**
14. *Causa:* Tu API de .NET bloqueó la petición de Angular por política de seguridad.
15. *Solución:* En `Program.cs` de .NET pon `app.UseCors("PermitirAngular");` antes de `app.MapControllers();`.

---

6. Parte 4: Cronograma de 90 Minutos para el Examen

Tiempo	Tarea	Verificación de Éxito
00 - 25 min	**Backend .NET:** Crear Modelos, DbContext, DA y Controller con CORS en `Program.cs`	Compilar sin errores en Visual Studio
25 - 35 min	**Swagger:** Ejecutar backend y probar `GET /api/Ejecuciones/Listar` y `GET /ListarPorId/1`	Devuelve `200 OK` con JSON
35 - 45 min	**Front Cimientos:** `provideHttpClient()` en `app.config.ts`, `product.model.ts` y servicio `ejecuciones.ts`	Servicio compila sin errores
45 - 75 min	**Front Componente:** Crear `panel.ts`, `panel.html` y `panel.css`. Vincular con `<app-panel>`	Clic en Products y la tabla muestra los datos
75 - 90 min	**Detalle y Pruebas:** Probar botón ojo (Ver Detalle), botón Volver y revisar consola F12	Todo fluye sin errores